MI SRC API

Version 1.2

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/12/2020
1.1	● Modified API	10/15/2020
1.2	● Add init parameter definition	06/01/2021

TABLE OF CONTENTS

1. Overview

1.1. Algorithm Description

SRC (short for Sample Rate Conversion) is used to convert the sampling frequency audio stream to obtain different sampling frequencies audio streams.

1.2. Keyword

- Sampling frequency: The number of samples sampled from a continuous signal per second.

1.3. Note

In order to facilitate debugging and confirm the effect of the algorithm, the user application needs to implement replacement algorithm parameter and grab audio data.

2. API Reference

2.1. API List

API name	**Features**
IaaSrc_GetBufferSize	Get the memory size required for Src algorithm running
IaaSrc_Init	Initialize Src algorithm
IaaSrc_Run	Src algorithm processing
IaaSrc_Release	Release Src algorithm resources

2.2. IaaSrc_GetBufferSize

- Features

  Get the memory size required for Src algorithm running.

- Syntax

  unsigned int IaaSrc_GetBufferSize(SrcConversionMode mode);

- Parameters

Parameter Name	Description	Input/Output
SrcConversionMode mode	Sample rate conversion type	Input

- Return value

  Return value is the memory size required for Src algorithm running.

- Dependency
  - Header: AudioSRCProcess.h
  - Library: libSRC_LINUX.so/ libSRC_LINUX.a

※ Note

The interface only returns the required memory size, and the application and release of memory need to be processed by the application.

- Example

  Please refer to IaaSrc_Run example.

2.3. IaaSrc_Init

- Features

  Initialize Src algorithm.

- Syntax

  SRC_HANDLE IaaSrc_Init(char *workingBufferAddress, SRCStructProcess *src_struct);

- Parameters

Parameter Name	Description	Input/Output
working_buffer_address	Memory address used by Src algorithm	Input
SRCStructProcess	Src algorithm initialization structure pointer	Input

- Return value

Return value	Result
Not NULL	Successful
NULL	Failed

- Dependency

- Header: AudioSRCProcess.h
- Library: libSRC_LINUX.so/ libSRC_LINUX.a

※ Note

None.

➢ Example

Please refer to IaaSrc_Run example.

2.4. IaaSrc_Run

➢ Features

Src algorithm processing.

➢ Syntax

ALGO_SRC_RET IaaSrc_Run(SRC_HANDLE handle, short *audio_input, short *audio_output, int* output_size);

➢ Parameters

Parameter Name	Description	Input/Output
handle	Src algorithm handle	Input
audio_input	Data pointer before resampling	Input
audio_output	Resampled output data pointer	Output
output_size	Resampled output data length. According to SRC mode and point_number, this data length will increase or decrease with the ratio of sampling rate change. Example: if the SRC mode is convert 8K to 48K, the output_size will be 6 times of point_number.	Output

➢ Return value

Return value	Result
0	Successful
非 0	Failed, refer to error code

➢ Dependency

- Header: AudioSRCProcess.h
- Library: libSRC_LINUX.so/ libSRC_LINUX.a

※ Note

- The buffer stored in audio_output needs to be passed in by the application
- Npoints range: 128, 512, 1024.

➢ Example

```
#include <stdio.h>
#include <stdlib.h>

#include "AudioSRCProcess.h"

/*  0:Fixed input file  1:User input file   */
#define IN_PARAMETER 1

int main(int argc, char *argv[])
{
    SRCStructProcess src_struct;
    /**********************User change section start*******************/
    /*
    *  Audio file bit wide:The user modifies as needed,
    *  for example:16, 8
    */
    int srcFileBitWide = 16;
    /*
    *  The user modifies as needed,for example:1, 2
    */
    src_struct.channel = 1;
    /*
    *  The user modifies as needed,for example:
    *  SRATE_8K, SRATE_16K, SRATE_32K, SRATE_48K
    */
    src_struct.WaveIn_srate = SRATE_8K;
    /*
    *  The user modifies as needed,for example:
    *  SRC_8k_to_16k, SRC_8k_to_32k, SRC_48k_to_8k ...
    */
    src_struct.mode = SRC_8k_to_48k;
    /*
    *  The user modifies as needed,for example:
    *  256, 512, 1024 and 1536 (Please select among these values)
    */
    src_struct.point_number = 1536;
    /**********************User change section end*******************/
    SrcConversionMode mode = src_struct.mode;
    SRC_HANDLE handle;
    ALGO_SRC_RET ret;
    int output_size;
    unsigned int workingBufferSize;
    char *workingBufferAddress = NULL;
    int freadBufferSize;
    char *freadBuffer = NULL;
```

```
46.     int fwriteBufferSize;
47.     char *fwriteBuffer = NULL;
48.     int nBytesRead;
49.     int nPoints = src_struct.point_number;
50.     int MaxFactor = SRATE_48K / SRATE_8K;//The biggest factor
51.     FILE* fpIn;  //input file
52.     FILE* fpOut; //output file
53.     char src_file[128] = {0};
54.     char dst_file[128] = {0};
55.
56.  #if IN_PARAMETER
57.     if(argc < 3)
58.     {
59.         printf("Please enter the correct parameters!\n");
60.         return -1;
61.     }
62.     sscanf(argv[1], "%s", src_file);
63.     sscanf(argv[2], "%s", dst_file);
64.  #else
65.     sprintf(src_file, "%s", "./8K_16bit_MONO_30s.wav");
66.     if(argc < 2)
67.     {
68.         printf("Please enter the correct parameters!\n");
69.         return -1;
70.     }
71.     sscanf(argv[1], "%s", dst_file);
72.  #endif
73.
74.     fpIn = fopen(src_file, "rb");
75.     if(NULL == fpIn)
76.     {
77.         printf("fopen in_file failed !\n");
78.         return -1;
79.     }
80.     printf("fopen in_file success !\n");
81.     fpOut = fopen(dst_file, "wb");
82.     if(NULL == fpOut)
83.     {
84.         printf("fopen out_file failed !\n");
85.         return -1;
86.     }
87.     printf("fopen out_file success !\n");
88.     //(1)IaaSrc_GetBufferSize
89.     workingBufferSize =  IaaSrc_GetBufferSize(mode);
90.     workingBufferAddress = (char *)malloc(sizeof(char) * workingBufferSize);
91.     if(NULL == workingBufferAddress)
92.     {
93.         printf("malloc SRC workingBuffer failed !\n");
94.         return -1;
95.     }
96.     printf("malloc SRC workingBuffer success !\n");
97.     //(2)IaaSrc_Init
98.     handle = IaaSrc_Init(workingBufferAddress, &src_struct);
99.     if(NULL == handle)
100.    {
101.        printf("SRC:IaaSrc_Init  failed !\n");
102.        return -1;
103.    }
104.    printf("SRC:IaaSrc_Init  success !\n");
105.    freadBufferSize = src_struct.channel * nPoints * srcFileBitWide / 8;
```

```
106.    freadBuffer = (char *)malloc(freadBufferSize);
107.    if(NULL == freadBuffer)
108.    {
109.        printf("malloc freadBuffer failed !\n");
110.        return -1;
111.    }
112.    printf("malloc freadBuffer success !\n");
113.    fwriteBufferSize = MaxFactor * freadBufferSize;
114.    fwriteBuffer = (char *)malloc(fwriteBufferSize);
115.    if(NULL == fwriteBuffer)
116.    {
117.        printf("malloc fwriteBuffer failed !\n");
118.        return -1;
119.    }
120.    printf("malloc fwriteBuffer success !\n");
121.#if 0
122.    /*Consider whether the input file has a header*/
123.    fread(freadBuffer, sizeof(char), 44, fpIn); //Remove the 44 bytes header
124.#endif
125.    while(1)
126.    {
127.        nBytesRead = fread(freadBuffer, 1, freadBufferSize, fpIn);
128.        if(nBytesRead != freadBufferSize)
129.        {
130.            printf("needBytes =%d      nBytesRead = %d\n", freadBufferSize, nBytesRead);
131.            break;
132.        }
133.        //(3)IaaSrc_Run
134.        ret = IaaSrc_Run(handle, (short *)freadBuffer, (short *)fwriteBuffer, &output_size);
135.        if(ret < 0)
136.        {
137.            printf("SRC:IaaSrc_Run  failed !\n");
138.            return -1;
139.        }
140.        //printf("ret = %d\n", ret);
141.        fwriteBufferSize = src_struct.channel * output_size * srcFileBitWide / 8;
142.        fwrite(fwriteBuffer, 1, fwriteBufferSize, fpOut);
143.    }
144.    fclose(fpIn);
145.    fclose(fpOut);
146.    //(4)IaaSrc_Release
147.    ret = IaaSrc_Release(handle);
148.    if(ret)
149.    {
150.        printf("IaaSrc_Release failed !\n");
151.        return -1;
152.    }
153.    printf("IaaSrc_Release success !\n");
154.    free(workingBufferAddress);
155.    free(freadBuffer);
156.    free(fwriteBuffer);
157.
158.    return 0;
159.}
```

2.5. IaaSrc_Release

- Features

Release Src algorithm resources.

- Syntax

ALGO_SRC_RET IaaSrc_Release(SRC_HANDLE handle);

- Parameters

Parameter Name	Description	Input/Output
handle	Src algorithm handle	Input

- Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

- Dependency
  - Header: AudioSRCProcess.h
  - Library: libSRC_LINUX.so/ libSRC_LINUX.a

※ Note

None.

- Example

Please refer to IaaSrc_Run example.

3. SRC Data Type

3.1. SRC data type list

DATA TYPE	**Description**
SrcConversionMode	Src algorithm sampling rate conversion type
SRCStructProcess	Src algorithm initialization parameter structure type
SrcInSrate	Src algorithm sampling rate type
SRC_HANDLE	Src algorithm handle type

3.2. SrcConversionMode

- Description

  Define Src algorithm sampling rate conversion type.

- Definition

```
typedef enum{
    SRC_8k_to_16k,
    SRC_8k_to_32k,
    SRC_8k_to_48k,
    SRC_16k_to_8k,
    SRC_16k_to_32k,
    SRC_16k_to_48k,
    SRC_32k_to_8k,
    SRC_32k_to_16k,
    SRC_32k_to_48k,
    SRC_48k_to_8k,
    SRC_48k_to_16k,
    SRC_48k_to_32k
}SrcConversionMode;
```

- Member

Member name	Description
SRC_8k_to_16k	8K resampling to 16K
SRC_8k_to_32k	8K resampling to 32K
SRC_8k_to_48k	8K resampling to 48K
SRC_16k_to_8k	16K resampling to 8K
SRC_16k_to_32k	16K resampling to 32K
SRC_16k_to_48k	16K resampling to 48K
SRC_32k_to_8k	32K resampling to 8K
SRC_32k_to_16k	32K resampling to 16K
SRC_32k_to_48k	32K resampling to 48K
SRC_48k_to_8k	48K resampling to 8K
SRC_48k_to_16k	48K resampling to 16K
SRC_48k_to_32k	48K resampling to 32K

※ Note

None.

- Related data types and interfaces

IaaSrc_GetBufferSize

3.3. SRCStructProcess

- Description

Define Src algorithm initialization parameter structure type.

- Definition

```
typedef struct{
    SrcInSrate WaveIn_srate;
    SrcConversionMode mode;
    unsigned int channel;
    unsigned int point_number;
}SRCStructProcess;
```

- Member

Member name	Description
WaveIn_srate	Sampling frequency of source data
mode	Sampling frequency conversion mode, please choose according to the sampling frequency of source data and output data.
channel	Source data channels
point_number	Input point number for SRC. The output_size of IaaSrc_Run will change according to the ratio of sampling rate conversion and point_number. For example: when point_number is 128 and SRC mode is SRC_8k_to_48k, output_size is $128 \times 6 = 768$. It is recommended that you should set the point number as your need. Setting value: [256, 512, 1024, 1536, 2880].

※ Note

None.

➢ Related data types and interfaces

IaaSrc_Init

3.4. SrcInSrate

➢ Description

Define Src algorithm sampling rate type.

➢ Definition

```
typedef enum{
   SRATE_8K  =  8,
   SRATE_16K =  16,
   SRATE_32K =  32,
   SRATE_48K =  48
}SrcInSrate;
```

- Member

Member name	Description
SRATE_8K	8K sampling frequency
SRATE_16K	16K sampling frequency
SRATE_32K	32K sampling frequency
SRATE_48K	48K sampling frequency

※ Note

None.

- Related data types and interfaces

SRCStructProcess

3.5. SRC_HANDLE

- Description

Define Src algorithm handle type.

- Definition

typedef void* SRC_HANDLE;

- Member

Member name	Description

※ Note

None.

- Related data types and interfaces

IaaSrc_Init

IaaSrc_Run

IaaSrc_Release

4. Error Code

SRC API error codes are shown as follow:

Table 4-1 SRC API error code

Error code	Definition	Description
0x00000000	ALGO_SRC_RET_SUCCESS	Successful
0x10000601	ALGO_SRC_INIT_ERROR	Not initialized
0x10000602	ALGO_SRC_RET_INVALID_MODE	Parameter setting is invalid
0x10000603	ALGO_SRC_RET_INVALID_HANDLE	HANDLE is invalid
0x10000604	ALGO_SRC_RET_INVALID_CHANNEL	Channel number doesn`t support
0x10000605	ALGO_SRC_RET_INVALID_POINT_NUMBER	Points per frame doesn`t support
0x10000606	ALGO_SRC_RET_INVALID_SAMPLE_RATE	Sampling frequency doesn`t support
0x10000607	ALGO_SRC_RET_API_CONFLICT	Other APIs are running
0x10000608	ALGO_SRC_RET_INVALID_CALLING	Incorrect order of calling API